

cycle-by-cycle trace

Part-A

I_1 : LD $F_2, 0(R_1)$

I_2 : mul F_4, F_2, F_6 ($F_4 = F_2 \times F_6$)

I_3 : ADD F_8, F_2, F_{10} ($F_8 = F_2 + F_{10}$)

I_4 : SUB F_4, F_8, F_2 ($F_4 = F_8 - F_2$)

I_5 : LD $F_8, 8(R_1)$

I_6 : ADD F_{12}, F_4, F_8 ($F_{12} = F_4 + F_8$)

load latency : 2 cycles

Add/sub latency : 2 cycles

mul latency : 6 cycles

(in trace, I am skipping fetch, decode stages)

load RS's

RS	Busy	op	V_j	V_K	Q_j	Q_K
----	------	----	-------	-------	-------	-------

load 1

load 2

ADD/SUB RS's

RS	Busy	op	V_j	V_K	Q_j	Q_K
ADD/SUB 1						
ADD/SUB 1						

MUL RS's

RS	Busy	op	V_j	V_K	Q_j	Q_K
MUL 1						
MUL 2						

Cycle - 1

RS	Busy	op	V_j	V_K	Q_j	Q_K
load 1	yes	LD	$[R_1]$	0	0	0
load 2						

RST : F_2 - load 1

CDB : nothing.

cycle - 2

R_s	Busy	op	V_j	V_k	Q_j	Q_k
load 1	yes (exe)	LD	$[R_1]$	0	0	0
mul 1	yes	mul	0	$[F_6]$	load 1	0

RST :

F_2 - load 1

F_4 - mul 1

CDB : nothing.

Cycle - 3

R_s	Busy	op	V_j	V_k	Q_j	Q_k
load 1	No					
mul 1	yes (exe)	mul	$[F_2]$	$[F_6]$	0	0
ADD 1	yes	ADD	$[F_2]$	$[F_{10}]$	0	0

RST :

F_8 - ADD 1

F_4 - mul 1

CDB : { load 1, load 1_value }

load 1 is completed it's execution
 so F_2 value is available.

cycle - 4

R_s	Busy	op	V_j	V_k	Q_j	Q_k
load 1	No					
mul 1	yes	(ex) mul	$[F_2]$	$[F_6]$	0	0
ADD 1	yes	(ex) ADD	$[F_2]$	$[F_{10}]$	0	0
ADD 2	yes	SUB	0	$[F_2]$	ADD 1	0

RST :

F_8 — ADD 1

F_4 — ADD 2

CDB : nothing

Cycle 5

R_s	Busy	op	V_j	V_k	Q_j	Q_k
load 1	yes	LD	[R1]	8	0	0
mul 1	yes (core) mul		[F5]	[F6]	0	0
ADD 1	NO					
ADD 2	yes	SUB	[ADD_1 Value]	[F2]	0	0

RST :

F_4 — ADD 2

F_8 — load 1

CDB : { ADD 1, ADD_1 value }

ADD 1 is completed it's execution

so F_8 data is available.

cycle 6

R_s	Busy	op	V_j	V_k	Q_j	Q_k
load 1	yes	(exe) LD	[R1]	8	0	0
mul 1	yes	(exe) mul	[F5]	[F6]	0	0
ADD 1	yes	ADD	0	0	ADD2	load 1
ADD 2	yes	(exe) SUB	[ADD_1 Value]	[F2]	0	0

RST :

F_{12} — ADD 1

F_4 — ADD 2

F_8 — load 1

CDB : no

cycle 7

R_s Busy op V_j V_k Q_j Q_k

load 1 no

mul 1 yes(exec) mul $[F_5]$ $[F_6]$ 0 0

ADD 1 yes ADD 0 $[F_8]$ ADD₂

ADD 2 yes(exec) SUB $[ADD_1_Value]$ $[F_2]$ 0 0

RST :

$F_{12} - ADD_1$

$F_4 - ADD_2$

CDB : { load 1, load 1_value }

How load 1, ADD₂,

available

{ I am giving preference
to load 1
then ADD₂ }

cycle 8

R_s	Busy	op	V_j	V_k	Q_j	Q_k
load 1	NO					
mul 1	yes	mul	$[F_5]$	$[F_6]$	0	0
ADD 1	yes	ADD	$[F_7]$	$[F_8]$		0
ADD 2	NO					

RST : F_2 — ADD 1

CDB : $\{ \text{Add}_2, \text{Add}_2_Value \}$

mul 1 is completed But
Bus is not free so, next
cycle it uses

Cycle 9

R_s	Busy	op	V_j	V_k	Q_j	Q_k
load 1	NO					
mul 1	NO					
ADD 1	yes	ADD	$[F_4]$	$[F_8]$		
ADD 2	NO					

RST : $F_{12} - \text{ADD 1}$

CDB : $\{ \text{mul 1}, \text{mul 1 value} \}$

after 1 cycle i.e. in
"cycle 10" all six instructions
were completed.

part B

take

I_2 mul F_4, F_2, F_8

I_4 sub F_4, F_8, F_2
from "part A"

in cycle 2 I_2 is Issued to
B and F_4 is mapped to mul1

in cycle 4 I_4 is issued to
RS and now F_4 is overwritten
to the ADD2

in cycle 7 ADD2 finishes
and Broadcast to RS, Register
file and F_4 is updated

in cycle 9 the result of

mul 1 is Broadcast to Rs,
Register file. now due to our
writing in cycle 4 the F₄
won't get updated.
thus "waw" is prevented.